

Reto · 500 Sombras de Alberto

FILEMAID

Arquitectura y ADRs

FACTURA



DECISIÓN



TRAZA



GULUC3M

Luis Daniel Casais Mezquida
Albert Giurgiu
Jorge Adrian Saghin Dudulea
Henry Díaz Bordón

[GITHUB.COM/GULUC3M/HACKSPAIN26](https://github.com/GULUC3M/HACKSPAIN26)

DOM 20 SEP 2026 · 11:00

1 ARQUITECTURA GENERAL DEL SISTEMA

La arquitectura se sostiene sobre tres principios:

- **Dos fases estrictamente separadas.** La **extracción** produce *features* (material bruto, sin interpretación) y el **parser** la interpreta en *campos*: candidatos con su extractor y confianza en, que nunca se colapsan en el almacén; sólo la regla que necesita un escalar elige uno, dejando constancia de cuál y por qué.
- **Decisión determinista y pura.** El motor consume campos + reglas y emite el resultado junto a los códigos de regla que lo produjeron y un *snapshot* de la configuración activa. Umbrales y frontera NO_PAGAR/ESCALAR son configuración, no código.
- **Trazabilidad e idempotencia.** Cada fase sella una fila de evidencia se cachea: un re-run nunca repite ni re-factura trabajo. El estado vive en la base de datos.

El flujo completo, con el almacén y la revisión humana alimentándose de la evidencia de cada fase:

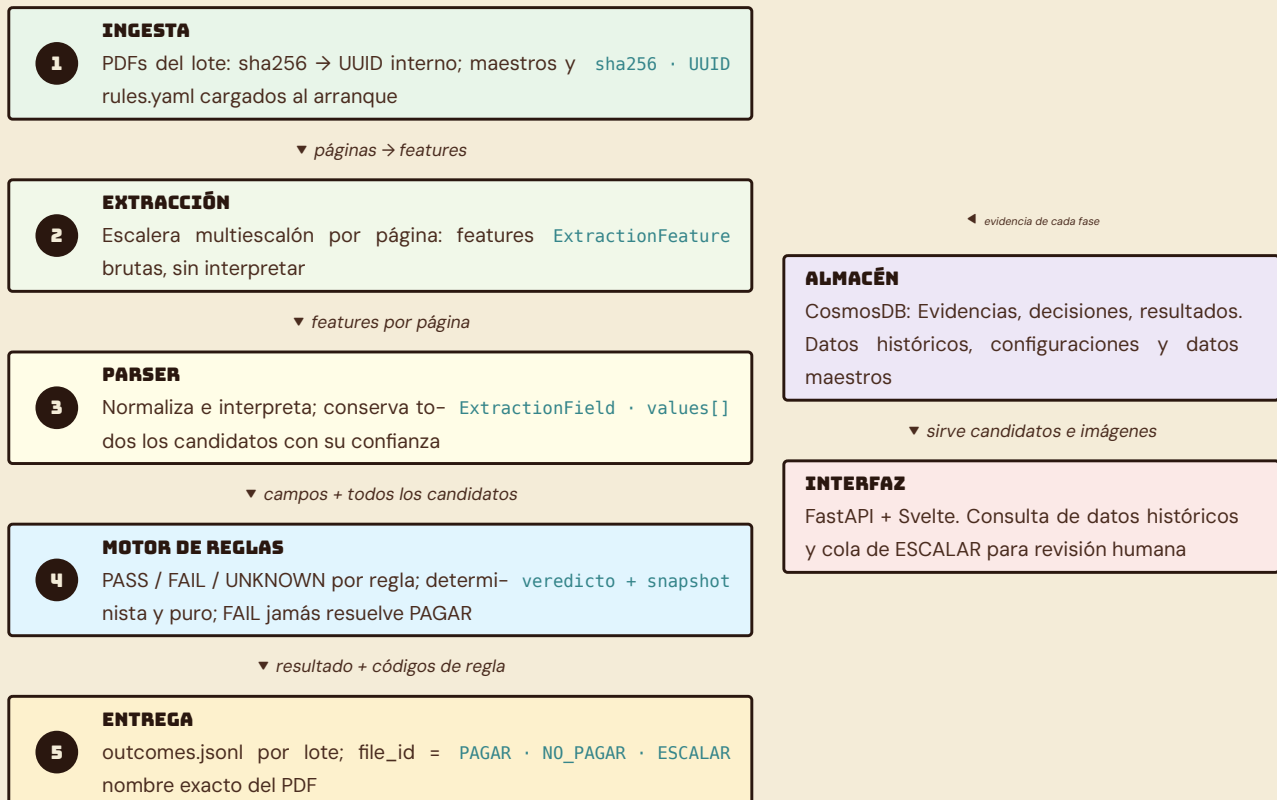


Figura 1 Arquitectura general

1.1 Bloque de extracción: La Escalera de Confianza Multiescalón

La extracción no es una caja negra: es una **escalera de 7 escalones calibrada por página**, diseñada para resolver al mínimo coste y escalar con rigor matemático ante cualquier anomalía documental.

1	Texto Vectorial (pypdf + Plausibilidad) Filtro ortográfico y alfanumérico anti-mojibake CID	< 1 MS 0 €
2	Rasterizado y QR (pypdfium2 + zxing) 300 DPI. Si la página es solo QR, payload = contenido	40 MS 0 €
3	Tesseract OCR (Doble Puerta) Pasa solo con word-conf > 40% y cobertura campos > 0.4	150 MS 0 €
4	VLM Local (PaddleOCR-VL 1.6 Q8 + OpenCV) Contexto 131k, KV q8_0, unsharp masking en escaneos	1.7 S 0 €
5	TypeSafe System One (Jev-latest) Decisiones tipadas y probabilidades paralelas (noul/choice/score)	560 MS 0.04 \$ / MTON
6	Firecrawl Document Parsing (/v2/parse) Rescate de tablas complejas y PDFs a Markdown estructurado	1.2 S 1 CREDIT
7	Cloud VLM (>25B Multimodal OpenAI comp.) Último recurso. Lectura como candidato extra (sin auto-stop)	3 S PAGO TOKEN

Figura 2 Escalera de confianza

1.1.1 Parser Resiliente Multilingüe

El *parser* consume las *features* y extrae candidatos preservando procedencia y confianza:

- **Inmune a trampas OCR:** Normaliza espacios invisibles (`\u200b`), separadores combinados (1 426,40, 1,135,20) y acrónimos fusionados (IVA21%27150).
- **Políglota nativo:** Reconoce importes y reglas fiscales en 7 idiomas (español, inglés, francés, alemán, italiano, catalán y portugués).
- **Desambiguación emisor/cliente:** Prioriza siempre el NIF del proveedor real sobre el CIF del cliente (Banco Miralmar), evitando falsos positivos de fraude.
- **Valores múltiples:** Todos los candidatos coexisten en `values[]`. La resolución escalar se difiere a la regla que la consume, garantizando trazabilidad y auditoría forense.

1.2 Motor de reglas y decisión

El motor consume los campos que entrega el *parser* (incluyendo todos sus candidatos) y, junto con los datos maestros y la configuración de la BBDD, produce la decisión: PAGAR, NO_PAGAR o ESCALAR. Es **determinista**: los mismos campos, con la misma configuración y datos, producen la misma salida.

1.2.1 Reglas

Cada regla vive en código y tiene un ID asociado. Añadir una regla no toca el bloque de extracción, y añadir un tipo de ficha no toca el motor.

Toda regla devuelve uno de tres veredictos, acompañado de un motivo legible y de los valores de campo que ha consumido:

- **PASS:** la regla se cumple.
- **FAIL:** la regla se incumple.
- **UNKNOWN:** no hay evidencia suficiente para decidir (campo ausente, lectura poco fiable, cruce no posible contra el maestro).

Las reglas definidas, extraídas del Excel `FINAL_v7_DEFINITIVO_ahorasi.xlsx`, son:

Tabla 1 Reglas vigentes

Código	Qué comprueba
NIF_IN_MASTER	El NIF del emisor está en el maestro de proveedores.
IBAN_MATCHES_MASTER	El IBAN de la factura coincide con el del maestro para ese NIF.
ORDER_BELONGS_TO_SUPPLIER	El pedido existe en el ERP, pertenece a ese proveedor y el importe de la factura cuadra con el suyo.
ORDER_PENDING	El estado del pedido en el ERP es PENDIENTE.
NO_DOUBLE_PAYMENT	El pedido no tiene ningún pago previo.
IVA_CONSISTENT	El IVA reportado coincide con $\text{base} \times \text{tipo} / 100$.
TOTALS_MUST_MATCH	El total coincide con $\text{base} + \text{IVA}$.
DATE_VALID_NOT_FUTURE	La fecha es válida y no es futura.

Las comparaciones de importes usan una tolerancia fija (0,01 EUR) para descartar falsos negativos por redondeo.

1.2.2 Veredictos

La frontera entre NO_PAGAR y ESCALAR es configuración, no código: un UNKNOWN es siempre duda razonable (escala), mientras que un FAIL resuelve al resultado configurado para esa regla: NO_PAGAR (negativo definitivo, valor por defecto) o ESCALAR (un humano debe revisarlo). Una regla rota nunca puede resolver PAGAR. Ante duda razonable, escalar antes que pagar.

Se tienen en cuenta los valores devueltos por las distintas reglas, y se sigue el siguiente algoritmo para realizar el veredicto:

1. Si hay algún FAIL, se comprueban las reglas que ha roto, y se aplica el *outcome* más restrictivo, dependiendo de la configuración (NO_PAGAR/ESCALAR).
2. Si alguna regla devuelve UNKNOWN, o no hay reglas que evaluar: ESCALAR.
3. Si no (todas PASS): PAGAR.

La salida del motor incluye, además del resultado:

- la lista completa de evaluaciones: código, veredicto, motivo, valores consumidos y candidato elegido para cada campo (con su extractor y confianza);
- un *snapshot* de la configuración activa: versión del conjunto de reglas, *thresholds*, versiones de los extractores y hash del maestro. Cualquier decisión se puede auditar y reproducir después.

1.2.3 Colapso de candidatos

Las reglas no consumen valores sueltos: consumen campos con todos sus candidatos. Cuando una regla necesita un escalar, el campo se colapsa:

1. `trim()` de los valores de texto.
2. Tests de formato con ID estable (NIF_FORMAT, IBAN_FORMAT, AMOUNT_POSITIVE, ...), seleccionables por campo desde la configuración: quien no los supera se rechaza por razón de formato.
3. A cada candidato se le asigna una puntuación, computada como la confianza por el peso del extractor. Cada campo tiene un *threshold* de puntuación que hay que superar para continuar.
4. Gana la puntuación máxima. En caso de empate, sólo si los valores no son iguales, se desempata por un ranking de extractores configurable por campo.

El proceso queda auditado candidato a candidato: cuál se eligió y por qué, y qué pasó con el resto (rechazado por formato, bajo el umbral, desfavorecido en el desempate). La elección se registra en

la evaluación de la regla. Si no sobrevive ningún candidato (campo ausente, todos rechazados por formato o por debajo del umbral), la regla devuelve UNKNOWN y el motor escala.



Figura 3 Arquitectura general

1.2.4 Configuración y maestros

Los *thresholds* y la selección de candidatos viven en la base de datos y pueden ser cargados a través de un YAML versionado cuyo hash forma parte del *snapshot*, con una entrada por código de regla:

Cada regla fija la confianza mínima exigida a los campos que consume; si el mejor candidato no la supera, la regla devuelve UNKNOWN en lugar de interpretar basura. Deshabilitar reglas o ajustar umbrales no requiere tocar código.

Los datos maestros (*proveedores.csv*, *pedidos.csv*) se cargan al arrancar y se almacenan en la base de datos. Consisten en:

- Listado de proveedores: NIF con su IBAN
- Pedidos del ERP con proveedor, importe, estado y pago. Su sha256 queda sellado en cada decisión, de modo que se sabe qué maestro estaba activo al

decidir; el cruce contra el ERP se hace a través de una interfaz intercambiable, sin acoplar el motor a ningún ERP concreto.

2

ADRS Y TRADE-OFFS

ADR 01 MOTOR DE REGLAS DETERMINISTA, CON REGLAS COMO CONFIGURACIÓN

CONTEXTO	La norma de negocio exige decisiones reproducibles y auditables: pagar solo si el NIF y el IBAN cruzan con el maestro, el pedido existe y cuadra, el IVA está bien calculado y la fecha es válida. Ante duda razonable, ESCALAR.
ALTERNATIVAS	LLM que decida directamente sobre el texto de la factura; reglas hardcodeadas acopladas a la extracción; reglas en YAML sin versionado ni snapshot.
DECISION	La decisión final la toma un motor de reglas en código, determinista y puro, pero configurable y modificable. Los umbrales de confianza son configuración versionada por campo y extractor; cada decisión guarda un snapshot (umbrales). Añadir una regla no toca extracción y viceversa; cambiar la política exige ADR.
CONSECUENCIAS	Determinismo y explicabilidad completos (cada resultado lista las reglas aplicadas y la configuración usada); el reprocesado tras un cambio de datos (regla v4, maestro actualizado) funciona desde el diseño, no como parche.

ADR 02 EXTRACCIÓN EN DOS BLOQUES: FEATURES Y PARSER

CONTEXTO	Las facturas llegan como PDF nativos y escaneados, con QRs en algunas legislaciones. Cada campo debe poder rastrearse hasta su origen en el documento. Los umbrales iniciales del <i>rung</i> 3 se fijaron SIN medición.
ALTERNATIVAS	Un único modelo end-to-end que devuelva los campos con trazabilidad; OCR monolítico con plantillas fijas por proveedor; mantener los umbrales a ojo.
DECISION	Bloque de extracción produce <i>features</i> tipadas (texto, tablas, QR, OCR) y un <i>parser</i> independiente las convierte en campos con valores múltiples, cada uno con extractor y nivel de confianza. Umbrales del <i>rung</i> 3 CALIBRADOS con el corpus real, elegidos en el hueco bimodal de las distribuciones medidas.
CONSECUENCIAS	Más piezas que operar, pero cada extractor es sustituible y comparable; el 94,2 % del corpus resuelve por texto (peldaño casi gratis) y el 89,7 % de las páginas OCR pasa el gate con los 3 escaneos ilegibles a revisión.

ADR 03 PIPELINE DESACOPLADA

CONTEXTO	El procesamiento por factura es independiente del resto: el problema es <i>embarassingly parallelizable</i> y hay que absorber lotes de archivos. El ERP del usuario está fuera del alcance del reto: solo se exige la costura de integración.
ALTERNATIVAS	Monolito secuencial; repartir en microservicios desde el inicio; integrar el ERP de verdad dentro del pipeline.
DECISION	Pipeline en Python con colas entre bloques de extracción, <i>parser</i> y decisión, de modo que cada bloque escale horizontalmente de forma independiente y un fallo del proveedor de LLM no bloquee el resto. El ERP se toca SOLO a través de una costura de adaptador; ningún módulo del pipeline conoce el ERP concreto.
CONSECUENCIAS	Infraestructura algo mayor que un script secuencial; a cambio, escalado por bloque, reintentos aislados y recuperación sin duplicados (idempotencia por <i>invoice id</i> y por (sha256, stage, versión)), y el día de mañana se conecta un ERP real sin tocar reglas ni extracción.

ADR 04 ALMACÉN DOCUMENTAL POUCHDB + REPLICACIÓN NATIVA A COUCHDB, EN LUGAR DE UNA BBDD RELACIONAL

CONTEXTO	El plan inicial era una BBDD relacional (SQLite local y Postgres en servidor, o equivalentes). Pero lo que el sistema conserva no encaja en tablas: los artefactos escaneados de cada documento (PDF original, PNG, intermedios), los logs semiestructurados de cada escalón de la escalera, el propio fichero, y además decisiones, candidatos y overrides con procedencia. La forma es heterogénea, anidada y evoluciona por documento; referenciar los binarios por ruta externa rompía la trazabilidad y la reanudación. El sistema corre 24/7 sin supervisión, debe sobrevivir a reinicios y a caídas de red, y el histórico debe poder replicarse entre dispositivos.
ALTERNATIVAS	SQLite relacional + ledger JSONL <i>append-only</i> (el diseño original); Postgres en servidor con tablas normalizadas; almacén de objetos (S3/MinIO) para binarios + SQL solo para metadatos; base documental embebida con replicación nativa (PouchDB local + CouchDB).
DECISION	Se adopta un almacén documental: PouchDB 9 embebido (adaptador LevelDB) como única persistencia <i>runtime</i> local, y un conector de replicación nativa contra una instancia CouchDB en la nube preexistente. Los binarios se guardan como adjuntos fragmentados (< 1 MiB) enlazados por documento, nunca por ruta externa; features, campos, decisiones, caché, eventos y overrides son documentos JSON con ID determinista, y los logs semiestructurados y la configuración local viven en el mismo motor. Los <i>exports</i> son proyecciones, no el almacén. La sincronización es la replicación nativa de CouchDB (revisiones, adjuntos y <i>checkpoints</i>), no un protocolo propio.

CONSECUENCIAS

Un único motor cubre documentos, binarios, logs y caché, y la sincronización local↔nube es replicación de serie, sin ETL ni esquema de migración. A cambio se renuncia a *joins*, restricciones y transacciones multi-documento: la consistencia se apoya en IDs deterministas, inmutabilidad y detección de conflictos *fail-closed* (nunca *last-write-wins*), y las consultas usan vistas versionadas por prefijo. Exige Node ≥ 20 y un puente acotado Python→JS. El histórico relacional anterior se conserva como enlace de compatibilidad, nunca como clave única de fichero.